

Optimizing single-track train timetables and route assignments in the city of João Pessoa

Renata Mendes Pereira Campos



CENTRO DE INFORMÁTICA
UNIVERSIDADE FEDERAL DA PARAÍBA

João Pessoa
2026

Renata Mendes Pereira Campos

Optimizing single-track train timetables and route assignments in the city of João Pessoa

Monograph presented in fulfillment of the requirements for the Bachelor's degree in Computer Engineering, Centro de Informática, Universidade Federal da Paraíba.

Advisor: Teobaldo Leite Bulhões Junior

João Pessoa

2026

Ficha catalográfica: elaborada pela biblioteca do CI.

Será impressa no verso da folha de rosto e não deverá ser contada.

Se não houver biblioteca, deixar em branco.



CENTRO DE INFORMÁTICA
UNIVERSIDADE FEDERAL DA PARAÍBA

Work titled *Optimizing single-track train timetables and route assignments in the city of João Pessoa* presented in fulfillment of the requirements for the Bachelor's degree in Computer Engineering, approved by:

Prof. Dr. Teobaldo Leite Bulhões Junior
Universidade Federal da Paraíba

Prof. Dr. Anand Subramanian
Universidade Federal da Paraíba

Prof. Dr. Luciano Carlos Azevedo da Costa
Universidade Federal da Paraíba

João Pessoa, October 17, 2025

*To my father,
for showing me that mathematics and engineering could be a path worth following.*

Acknowledgments

I would like to sincerely thank my advisor, Prof. Teobaldo Bulhões, for his guidance and support throughout this work. I am also grateful to Prof. Anand Subramanian and Prof. Bruno Bruck for their valuable contributions during the development of this research. The advice and expertise of all three were essential for the completion of this project, and I am deeply grateful for everything I have learned from them. I would also like to thank Prof. Luciano Costa for composing the examining committee alongside Prof. Teobaldo and Prof. Anand.

I am thankful to my parents, Rachel Mendes and Manuel Campos, for their continuous support and investment in my education. I am especially grateful to my father for encouraging my interest in mathematics ever since I was a child. I would also like to thank my brother, Roberto Mendes, for introducing me to the fields of programming and computer science. Their support and encouragement have been essential in shaping the professional I am becoming.

I am grateful to Guilherme Jácome for being a constant presence and source of support in my life, both personally and throughout my undergraduate studies, encouraging me during challenging moments, keeping me focused on my goals, and bringing joy to my everyday life.

Finally, I would like to thank all the friends I made during my undergraduate years, as well as my colleagues in the laboratory. I have learned a lot from each of them, and their presence has made the daily routine more enjoyable.

Resumo

Este trabalho é resultado de uma colaboração com uma empresa ferroviária brasileira, que opera em uma linha de trens composta por um único trilho, em que os veículos precisam trafegar em sentidos opostos simultaneamente. Assim, o trabalho aborda uma variante do Problema de Programação de Horários de Trens, em que horários de chegada e saída devem ser determinados, ao mesmo tempo que rotas predefinidas devem ser atribuídas aos veículos. O objetivo é minimizar o tempo de espera dos passageiros nas estações. Para resolver o problema, foi proposta uma formulação matemática, juntamente com um método exato baseado na enumeração de rotas e um algoritmo heurístico baseado em Programação Linear Inteira Mista. Os métodos propostos foram capazes de encontrar boas soluções mesmo para instâncias complexas baseadas em cenários do mundo real.

Key-words: Programação de horários de trens, Escalonamento, Programação inteira.

Abstract

This work results from a collaboration with a Brazilian train company that operates a single-track railway network, where trains can travel in both directions simultaneously. It addresses a variant of the Train Timetabling Problem, where arrival and departure times must be determined, while predefined routes must be assigned to the vehicles. The objective is to minimize passengers' waiting times at the stations. To address this, a mathematical formulation was proposed, along with an exact method based on routes enumeration and a MILP-based heuristic algorithm. The proposed methods were able to find good solutions even for complex instances based on real-world scenarios.

Key-words: Train timetabling, Scheduling, Integer programming.

Contents

Glossary	v
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Preliminaries	1
1.2 Motivation	1
1.3 Objectives	2
1.4 Monograph outline	2
2 Literature review	3
2.1 Fundamental concepts and terminology	3
2.2 Related work	6
2.3 Literature positioning	9
3 Problem description	11
4 Methodology	13
4.1 Mathematical formulation	13
4.1.1 Sets	13
4.1.2 Data	17
4.1.3 Decision variables	17
4.1.4 Objective function and constraints	18
4.2 Enumeration of routes	21
4.3 MILP-based heuristic	22
5 Computational experiments	27
5.1 Instances	27
5.2 Results	28

6	Concluding remarks	34
	Bibliography	34

Glossary

CBTU	:	Companhia Brasileira de Trens Urbanos
ILP	:	Integer Linear Programming
MILP	:	Mixed-Integer Linear Programming
TTP	:	Train Timetabling Problem
TPP	:	Train Pathing Problem
TRP	:	Train Routing Problem

List of Figures

1	Graph of a cyclic timetable: the same pattern of trips is repeated periodically along the day.	3
2	Graph of a non-cyclic timetable: trips do not follow a repeated pattern along the day.	4
3	Railway network with multiple corridors.	4
4	Individual corridor.	4
5	Representation of the headway, defined as the time interval between two consecutive train departures from the same stopping point in the same direction.	6
6	Example of a passing loop on a single-track railway.	6
7	Representation of the real-life network as the undirected path graph G . . .	13
8	Representation of the network considered by the model as the directed graph G'	14
9	Quadruple $(k, q, v, a) \in C$, with v at crossing p_2 and k at crossing p_5	15
10	Quadruples $(k, q, v, a) \in C$, with v at crossing p_5 and k at crossing p_2	16
11	Set of matrices representing route combinations. Matrix (a) shows the baseline combination, while matrices (b), (c), and (d) show subsets generated from it.	23
12	Real instance: t3p16r6h1	32
13	Real instance: t3p16r6h6	32
14	Real instance: t4p16r6h1	33
15	Real instance: t4p16r6h6	33

List of Tables

1	Comparative summary of works in the literature.	9
2	Instances from 5 to 9 stopping points.	29
3	Instances from 10 to 14 stopping points.	29
4	Instances from 15 to 19 stopping points.	30
5	Instances from 20 to 24 stopping points.	30
6	Real-world CBTU instances.	31

1 Introduction

1.1 Preliminaries

The operational planning of a railway line is essential for its efficient functioning. In public transport, it is important to ensure that planning aligns with customer preferences, providing convenient arrival and departure times. At the same time, operational aspects must be carefully considered, with strict safety protocols followed to ensure a secure mode of transportation for all customers. Defining the itinerary of the vehicles, including their routes and timetables, can be a challenging task. This challenge can become even greater when efforts are made to minimize costs or enhance passengers' satisfaction. Therefore, effective techniques are needed to address this issue efficiently and reliably.

The Train Timetabling Problem (TTP), also referred to as the Train Scheduling Problem, aims to determine an optimal timetable for a train line, while considering all operational requirements of a railway company, such as distances between stopping points, stations' demands, dwell times, safety protocols, among others. In this work, we focus on a practical case from Companhia Brasileira de Trens Urbanos (CBTU), a state-owned company that manages a single-track railway line in the city of João Pessoa, in the state of Paraíba, Brazil. In addition to timetabling, this work also addresses the assignment of predefined route patterns to trains, which extends the traditional TTP into a more complex variant that has not yet been addressed in the literature.

1.2 Motivation

This study is motivated by the operational challenges faced in CBTU's railway system. The company's railway line consists of a single track, where trains must run in both directions at the same time. This makes the timetabling even more challenging, since train conflicts must be avoided. Creating a daily timetable by hand is reportedly a tiring task that becomes even harder as the number of trains in operation increases.

The CBTU train line is the only urban railway service operating in the state of Paraíba. The line covers a distance of 30 kilometers and carries about 7,000 passengers per day. It is an important transportation service for the local population, and the itinerary planned for the trains can directly affect their daily routines. Thus, efficiently solving this problem can represent a positive social impact on the lives of these people.

In this work, we propose methods to address this complex real-world problem, aiming to find (near-)optimal solutions that can be applied directly to the railway line.

1.3 Objectives

The objective of this work is to develop methods to efficiently solve a new variant of the TTP that incorporates simultaneous optimization of timetables and route assignments, considering the operational requirements of CBTU's railway system.

The specific objectives are described as follows.

- Introduce a new variant of the TTP that considers route assignments alongside timetabling.
- Provide a mathematical formulation based on mixed-integer linear programming (MILP) capable of addressing the proposed variant.
- Present an exact method based on the enumeration of route assignments.
- Develop a MILP-based heuristic approach to solve instances that cannot be efficiently handled by the exact methods, including real-world instances.
- Perform computational experiments to measure the effectiveness of the proposed methods.

1.4 Monograph outline

The remainder of this work is organized as follows. Section 2 presents a brief literature review. Section 3 defines the problem addressed in this work. Section 4 describes the proposed solution methods, including the exact and heuristic approaches. Section 5 contains the computational results. Finally, Section 6 concludes.

2 Literature review

2.1 Fundamental concepts and terminology

The definition of the TTP depends mainly on the demands of a railway company. For that reason, many variants have been studied over the years. In what follows, we define some important features that are typically considered when solving a TTP (Cacchiani and Toth, 2012).

- **Cyclic (periodic) timetable:** A cyclic timetable repeats itself at regular intervals throughout the day. Its timetabling typically involves a shorter planning horizon, since they follow a repeating pattern. Figure 1 shows a graph representing a cyclic timetable. The x -axis represents the time horizon, and the y -axis represents the stopping points along a corridor. The graphs illustrate the timetable for a scenario with two trains and five stopping points, with each train represented by a different color. This type of graph is commonly used in the context of railway systems, and allows us to determine the position of each train at any given time, helping to detect potential conflicts in one-way traffic and aiding timetable planning.

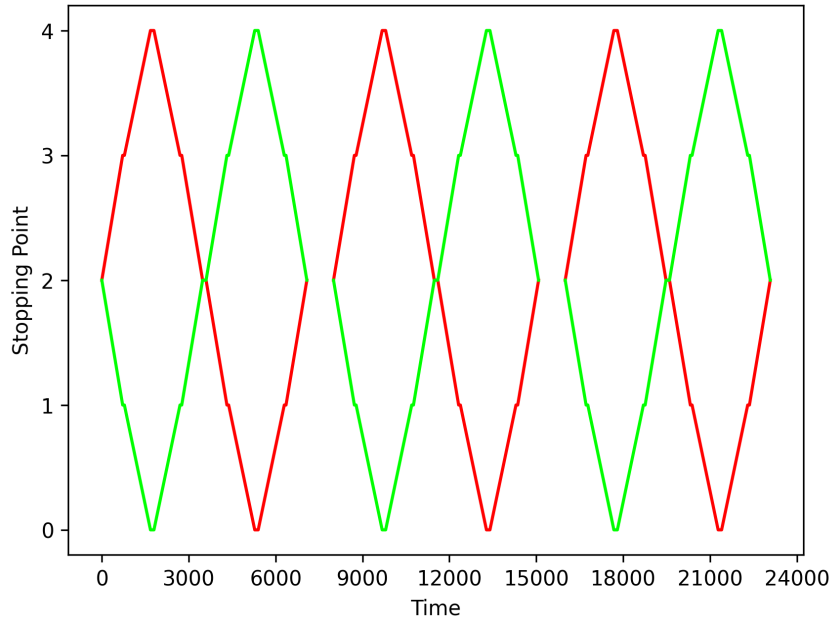


Figure 1: Graph of a cyclic timetable: the same pattern of trips is repeated periodically along the day.

- **Non-cyclic timetable:** A non-cyclic timetable does not follow a repeating pattern and may vary across the day. They usually require a longer and more flexible planning horizon. Similarly to Figure 1, Figure 2 shows a graph representing a non-cyclic timetable.

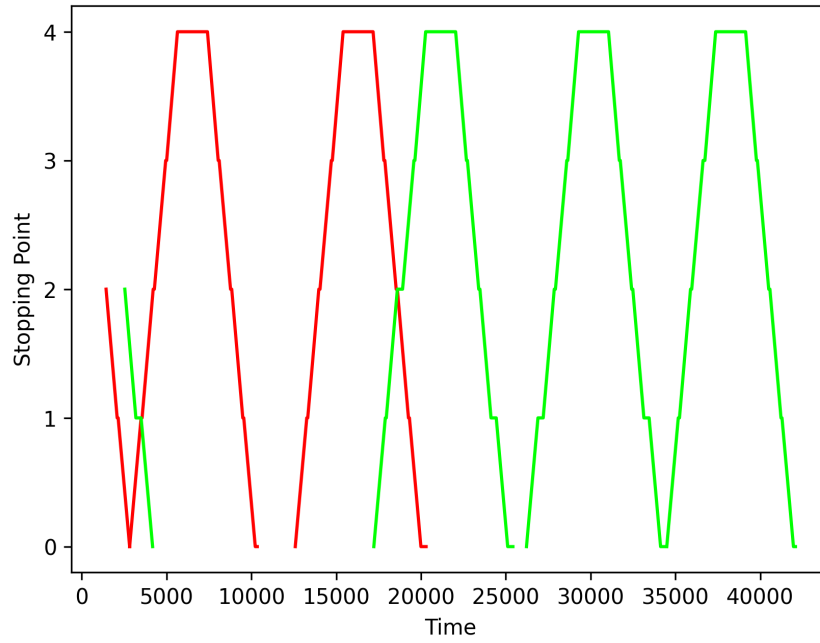


Figure 2: Graph of a non-cyclic timetable: trips do not follow a repeated pattern along the day.

- **Corridor:** A corridor is a sequence of stopping points connecting two major stations at its endpoints. Railway networks are often composed of multiple corridors. Many studies prefer to address the timetabling of individual corridors rather than the entire network, in order to reduce problem complexity. Figure 3 illustrates a railway network composed of four distinct corridors, each represented by a different color. Figure 4 depicts an individual corridor. In both figures, the corridors are represented as sequences of stopping points, with two major stations located at their endpoints.

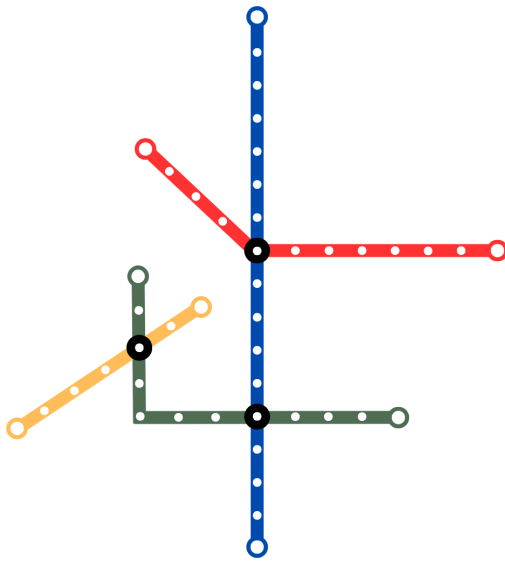


Figure 3: Railway network with multiple corridors.



Figure 4: Individual corridor.

- **Number of tracks:** Refers to the physical railway infrastructure available in a corridor. A corridor can be single-track, double-track, or even multi-track.
- **Two-way traffic:** A situation where trains must share the same track section to travel in opposite directions. This configuration requires careful timetabling to avoid collisions between trains and typically occurs in single-track corridors.
- **One-way traffic:** A situation where each direction of travel has a dedicated track, so trains do not interfere with opposing flows. This configuration is common in double-track or multi-track corridors, making the planning of the timetable generally easier.

Furthermore, the TTP can be adapted to address either freight or passenger transportation, an aspect that can directly affect the choice of the objective function. We list below some common objective functions that are frequently considered in the literature.

- **Minimize waiting times:** Reduce the time passengers spend waiting at stations.
- **Minimize dwell times:** Reduce the time trains spend stopped at stopping points.
- **Minimize travel times:** Reduce the total traveling time of trains along their routes.
- **Minimize costs:** Reduce the operational costs for the railway company, mainly associated with infrastructure usage, vehicles, and train crews.
- **Minimize penalties / Maximize profits:** Given a preferred timetable, penalties or profits are calculated based on how similar the generated timetable matches the preferred one.

Having discussed above the main aspects that influence TTP formulations and solutions, we now introduce key terminology commonly used in railway systems. These terms provide a foundation for better understanding the works mentioned in the following subsection (2.2).

- **Dwell time:** The time a vehicle stays at a stopping point.
- **Headway:** The time interval between consecutive departures from the same stopping point in a given direction. Figure 5 illustrates the headway between two trains, where Train 2 has already departed from the station, and Train 1 has not. If Train 1 were to depart at the instant shown in the figure, following the same direction as Train 2, the line highlighted in red represents the headway between the two trains.

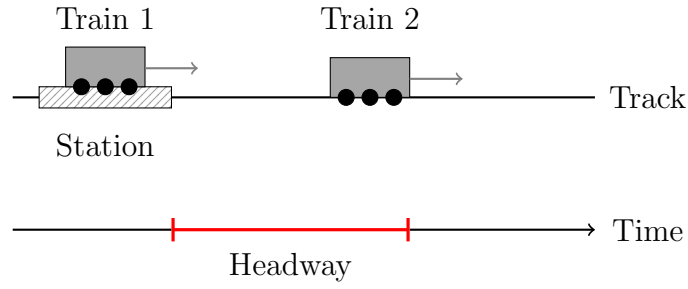


Figure 5: Representation of the headway, defined as the time interval between two consecutive train departures from the same stopping point in the same direction.

- **Track capacities:** The maximum number of trains that can occupy a track section or station within a given time interval.
- **Conflict:** A situation in which two or more vehicles require the same track section at the same time. In real operations, these conflicts may lead to collisions if not properly managed.
- **Crossing:** A situation in which one train passes another train traveling in the opposite direction.
- **Overtaking:** A situation in which one train passes another train traveling in the same direction.
- **Passing loops / Sidings:** Short sections of tracks connected to the main line, providing additional pathways for trains to cross or overtake each other. Figure 6 illustrates a passing loop on a single-track railway from a top view. In this example, the track splits into two branches, thus, if a train is stopped at point A, another train can overtake or cross by passing through point B.

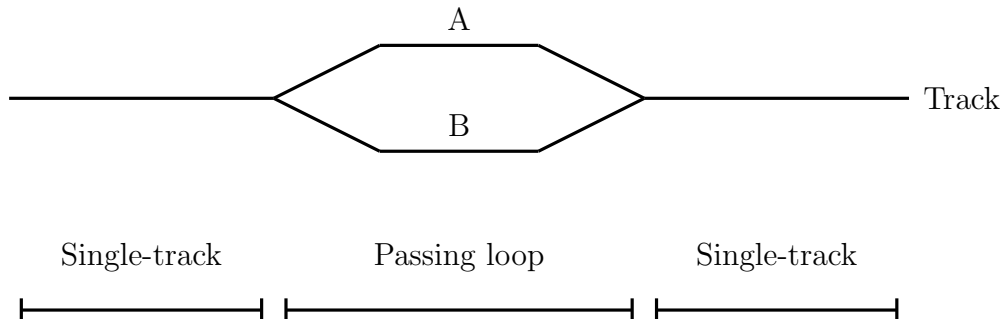


Figure 6: Example of a passing loop on a single-track railway.

2.2 Related work

Szpigel (1973) was one of the first to address the TTP. The author models a single-

track railway as a job-shop scheduling problem, where jobs can be seen as train trips and machines as railway tracks. The objective is to minimize trains' travel times, and the problem is solved using an exact branch-and-bound method.

Cai and Goh (1994) propose a greedy heuristic for the TTP on a single-track corridor, based on a MILP formulation where binary variables indicate whether a train stops in passing loops, and continuous variables represent the waiting time a train spends at these points. The approach aims to minimize total costs associated with train stops and delays while ensuring feasibility by resolving crossing and overtaking conflicts.

Carey and Lockwood (1995) deal with the scheduling of a train line with one-way traffic. They present a MILP formulation, using integer variables to indicate precedence between trains and continuous variables to represent arrival and departure times. The objective function aims to minimize deviations from a given preferred timetable. To solve the problem, the authors apply an iterative decomposition into subproblems, which are solved with heuristics and branch-and-bound. Higgins et al. (1997) develop a similar MILP formulation, with the objective of minimizing travel times while avoiding crossing conflicts and overtakes. Additionally, they implement heuristic approaches to solve the problem.

Brännlund et al. (1998) propose an Integer Linear Programming (ILP) model that divides the tracks into blocks, using binary variables to indicate whether a train occupies a block at a certain time instant, while Caprara et al. (2002) represent the problem on a time-space graph. In both cases, a Lagrangian relaxation approach is applied to solve the models, with the objective of maximizing profits.

Zhou and Zhong (2007) develop a MILP formulation that uses continuous variables to define arrival and departure times and binary variables to determine precedences, similar to Carey and Lockwood (1995) and Higgins et al. (1997). They also define binary variables to indicate whether a train occupies a segment at a certain time instant, similar to Brännlund et al. (1998). They solve the problem with branch-and-bound algorithms, aiming to minimize total train travel time on a single-track railway.

Now, focusing our attention on studies that prioritize improving passenger services, we highlight the following works. Vansteenwegen and Oudheusden (2006) introduce an objective function that minimizes an overall generalized waiting cost, considering deviations from ideal buffer times, waiting at stations, and transfer times, solved through a three-phase approach combining buffer calculation, linear goal programming, and simulation. Liebchen (2006) minimizes both transfer waiting times and dwell times using a branch-and-cut algorithm, constraint programming, and genetic algorithms. Both works deal with cyclic timetables. Moreover, Barrena et al. (2014) put forward methods that consider dynamic demands over a day while minimizing passengers' average waiting times

at stations in a scenario with a double-track train line. They develop three mathematical formulations and implement a branch-and-cut algorithm to solve them. Robenek et al. (2016) simultaneously consider profit maximization for operators and satisfaction maximization for passengers using a MILP model combined with an ε -constraint approach to handle the bi-objective problem, without accounting for train conflicts or track capacity constraints.

Furthermore, there is a good number of works that tackle the TTP integrated with the optimization of other features, some of which are listed ahead. Continuing with the objective of providing better timetables for passengers, Schmidt and Schöbel (2015) minimize passengers' routes when scheduling the timetable. They develop an efficient enumeration method that considers "virtual routings" and then solve the timetabling for each of these to find the optimal solution.

The Train Pathing Problem (TPP) and the Train Routing Problem (TRP) are commonly addressed along with the TTP, both consisting mainly of determining the tracks that trains need to take to arrive at their destinations while avoiding conflicts with other vehicles. Most of these problems aim to minimize trains' travel times. Some examples of studies that approach both train timetabling and pathing/routing are Carey and Lockwood (1995), which was already previously mentioned, Lee and Chen (2009), and Sun et al. (2014). Lee and Chen (2009) propose a heuristic that simultaneously integrates train pathing and timetabling, accounting for track-dependent operation times, track preferences, and minimum headways. Their method relies on an iterative four-step procedure combining neighborhood search and mathematical programming, and the objective is to minimize deviations from a given timetable. Sun et al. (2014) develop a multi-objective model for train routing and timetabling on high-speed railway networks, aiming to minimize average travel time, energy consumption, and delays, which is solved using a genetic algorithm.

Some studies address the TTP integrated with stop planning, i.e., deciding at which stations trains must stop, given the origin and destination of their trips. Yue et al. (2016) propose a solution for a scenario with high-speed trains, aiming to find the optimal timetable and stopping patterns for a one-way corridor. Their objective function considers penalties for stopping times and the number of stops, and the problem is solved using a column-generation-based heuristic algorithm with Lagrangian relaxation. More recently, Cacchiani et al. (2020) address a high-speed railway line while accounting for uncertain passenger demands, using robust MILP models based on the Light Robustness technique. In parallel, Boroun et al. (2020) propose a joint optimization of train timetabling and stop planning for double-track railways, aiming to minimize the arrival time of the last train, and solves the problem using MILP formulations combined with heuristic methods.

For a more detailed review of the TTP, Cacchiani and Toth (2012) provide an

extensive survey of studies on this topic. In addition, Parbo et al. (2016) review the literature and compare train-oriented timetabling approaches with passengers’ real perceptions and needs, showing that most studies focus on train performance and do not consider passengers’ experiences.

2.3 Literature positioning

It is important to emphasize that the current work aims to optimize a non-cyclic timetable for a single-track corridor, with two-way traffic, while simultaneously determining the sequence of stations visited on each train trip, in order to fulfill daily demands at each station.

At first, it may seem odd to optimize routes taken by trains along a single-track corridor since it involves a single line of stations. However, the CBTU railway system has major stations between the line’s endpoints that allow trains to change the direction in which they are traveling. Therefore, we must decide whether a train will keep going forward to the next station, reverse direction or even terminate the trip. Hence, this creates multiple possible routes that trains may complete along the corridor. Besides, we clarify that the routes referred to in the current work do not function the same way as those in problems such as TPP and TRP. In both of these, the objective is related to assigning tracks so the trains can reach their destination, which is already given as input. In the current work, the origins and destinations of the trains are not established in advance, and all trains must stop at every station they eventually visit. Furthermore, a more detailed description of the problem will be discussed in the following section (3).

Finally, even with the large number of works developed on the TTP over the past decades, no previous study has simultaneously addressed all the features considered in the present work. Table 1 provides a comparative summary of the works discussed in this section, highlighting their solving methods, traffic and timetable types, additional features, and objective functions, and emphasizing how the current study differs from the previous ones.

Table 1: Comparative summary of works in the literature.

Reference	Solving methods	Traffic	Timetable	Additional features	Objective function(s)
(Szpigel, 1973)	Branch-and-bound.	Two-way.	Non-cyclic.	–	Minimize travel times.
(Cai and Goh, 1994)	Heuristic.	Two-way.	Non-cyclic.	–	Minimize costs associated with train stops and delays.
(Carey and Lockwood, 1995)	Heuristics and branch-and-bound.	One-way.	Non-cyclic.	Train pathing.	Minimize deviations from preferred timetable.
(Higgins et al., 1997)	Heuristic.	Two-way.	Non-cyclic.	–	Minimize travel times.

Reference	Solving methods	Traffic	Timetable	Additional features	Objective function(s)
(Brännlund et al., 1998)	Lagrangian relaxation.	Two-way.	Non-cyclic.	–	Maximize profits given a preferred timetable.
(Caprara et al., 2002)	Lagrangian relaxation.	Two-way.	Cyclic.	–	Maximize profits given a preferred timetable.
(Zhou and Zhong, 2007)	Branch-and-bound.	Two-way.	Non-cyclic.	–	Minimize travel times.
(Vansteenwegen and Oudheusden, 2006)	Linear goal programming and simulation.	Two-way.	Cyclic.	–	Minimize waiting times.
(Liebchen, 2006)	Branch-and-cut, constraint programming, and genetic algorithms.	Two-way.	Cyclic.	–	Minimize waiting and dwell times.
(Barrena et al., 2014)	Branch-and-cut.	One-way.	Non-cyclic.	Dynamic demands.	Minimize waiting times.
(Robenek et al., 2016)	ε -constraint technique.	One-way.	Cyclic and non-cyclic.	–	Maximize profit for operators and maximize satisfaction for passengers.
(Schmidt and Schöbel, 2015)	Enumeration of virtual routings.	One-way.	Non-cyclic.	Passengers' routings.	Minimize passengers' route.
(Lee and Chen, 2009)	Heuristic.	Two-way.	Non-cyclic.	Train pathing.	Minimize deviations from preferred timetable.
(Sun et al., 2014)	Genetic algorithm.	One-way.	Non-cyclic.	Train routing.	Minimize average travel time, energy consumption, and delays.
(Yue et al., 2016)	Lagrangian relaxation.	Two-way.	Non-cyclic.	Stop planning.	Minimize penalties for stopping times and the number of stops.
(Cacchiani et al., 2020)	Light Robustness technique.	Two-way.	Non-cyclic.	Stop planning.	Minimize travel times or the minimization of the unsatisfied demand or unavailable capacity.
(Boroun et al., 2020)	Mathematical formulations with heuristic.	One-way.	Non-cyclic.	Stop planning.	Minimize the arrival time of the last train.
Current study	Mathematical programming, routes enumeration and heuristic.	Two-way.	Non-cyclic.	Route assignments.	Minimize waiting times.

3 Problem description

The problem addressed in this work consists of scheduling a train timetable and assigning route patterns to the vehicles operating on a train line, based on the case of CBTU, in the city of João Pessoa, Brazil. The CBTU railway system consists of a single-track corridor, where trains can travel in both directions simultaneously. Intermediate stopping points are distributed along the line, connecting its two endpoints. The stopping points can be classified into three categories: stations, crossings, and depots.

- Stations are points where passengers can board and alight from the vehicles.
- Crossings are points that allow trains traveling in opposite directions to pass each other. They correspond to passing loops along the line.
- Depots are points where trains can start and end their trips. They also allow trains to reverse their direction of travel. Depots can be considered the major stations of the train line. By definition, the endpoints must be depots, but some intermediate stations can also be classified as such. Also, it is important to note that every depot is also a crossing and a station. However, not every point classified as station and crossing is necessarily a depot.

Since some intermediate points can serve as depots, allowing trains to start and end trips as well as reverse their direction, it is possible for trains to perform different routes along the line. Therefore, we are given a set of predefined routes that must be assigned to each trip made by the trains.

Each train is given a maximum number of trips that it can complete in a single day. Each trip completed by a train must be assigned to a single route. Moreover, all trains must start their first trip of the day at the same depot, referred to as the initial depot. Furthermore, a series of operational constraints must be taken into account in order to guarantee the safety of train operations, as well as to meet the demands of the railway company. These constraints are detailed as it follows.

- Minimum and maximum dwell times at stopping points must be respected.
- All trips of the day must be finished before the maximum operating time of the train line is reached.
- The minimum headway between train departures must be respected.
- Trains cannot overtake each other when traveling in the same direction.

- When traveling in opposite directions, trains can only pass one another at crossing points, otherwise, a collision would happen.
- Each station has a demand for trains that must stop there during specific time intervals. This allows us to account for rush hours, which may require more trains than other periods, as well as directional passenger flows that may concentrate toward one side of the city at certain times and reverse at others. In this way, we incorporate realistic demand patterns. It is relevant to point out that the demands are based on the departure direction of each train. Consequently, at the endpoints of the line, the demand in one of the directions must be set to zero, since trains cannot proceed beyond the terminal depot.

With these constraints in mind, the current study aims to improve passenger satisfaction. Many different objective functions can be developed to this end, as discussed in Section 2. In this work, we intend to minimize passenger waiting times at stations. Hence, we define the objective function as the minimization of the maximum time interval between departures from a station in a given direction. In this way, it is possible to minimize the time a person may wait at a station until the next train arrives.

4 Methodology

In this section, we describe the three distinct methods to address the problem: a mathematical formulation (Section 4.1), an exact method based on routes enumeration (Section 4.2), and a heuristic approach (Section 4.3).

4.1 Mathematical formulation

As a first step, we developed a mathematical formulation capable of representing the described problem.

4.1.1 Sets

Initially, we consider the following sets.

- T – set of trains.
- I_t – set of indices of trips that train $t \in T$ can perform, where $I_t = \{1, 2, \dots, i_t\}$ and i_t denotes the maximum number of trips that the train can complete in a single day.
- P – set of stopping points, with $P = \{p_1, p_2, \dots, p_n\}$.
- S – set of stations $S \subseteq P$.
- X – set of crossings $X \subseteq P$.
- D – set of depots $D \subseteq S \cap X$.

The problem is defined over a graph G , that is defined as follows. The railway line in real life can be defined as an undirected path graph $G = (P, E)$, where P is the set of nodes, and E is the set of edges connecting consecutive stopping points, with $E = \{\{p_1, p_2\}, \{p_2, p_3\}, \dots, \{p_{n-1}, p_n\}\}$. Figure 7 shows the representation of the railway network as graph G , where trains can travel in both directions on a single track.

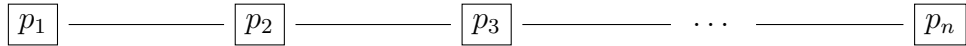


Figure 7: Representation of the real-life network as the undirected path graph G .

However, to simplify the development of the formulation, the directed graph $G' = (V, A)$, shown in Figure 8, was adopted. G' represents a modified version of the original network, where each stopping point $p \in P$ is now represented by two distinct vertices, each belonging to a different section of the network: upper or lower. The upper section

represents the flow of vehicles traveling from left to right, while the lower section corresponds to the flow of vehicles moving from right to left. Moreover, some arcs, such as $(\vec{v}_1, \overleftarrow{v}_1)$ and $(\overleftarrow{v}_1, \vec{v}_1)$, connect the upper and lower vertices of the points, representing depots. These are referred to as reversion arcs and represent the possibility of a train reversing its direction of travel.

Thus, we define the sets that compose the graph G' .

- $\vec{V} = \{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n\}$ – set of vertices of the upper section of the network.
- $\overleftarrow{V} = \{\overleftarrow{v}_1, \overleftarrow{v}_2, \dots, \overleftarrow{v}_n\}$ – set of vertices of the lower section of the network.
- $V = \vec{V} \cup \overleftarrow{V}$ – set of all vertices.
- $V_p = \{\vec{v}_p, \overleftarrow{v}_p\}$ – set of vertices that belong to point $p \in P$.
- $\vec{A} = \{\vec{a}_p = (\vec{v}_p, \vec{v}_{p+1}) : 1 \leq p < n\}$ – set of arcs of the upper section of the network, oriented from left to right.
- $\overleftarrow{A} = \{\overleftarrow{a}_p = (\overleftarrow{v}_{p+1}, \overleftarrow{v}_p) : 1 \leq p < n\}$ – set of arcs of the lower section of the network, oriented from right to left.
- $\overleftrightarrow{A} = \bigcup_{p \in D} \{\overleftrightarrow{a}_p = (\vec{v}_p, \overleftarrow{v}_p), (\overleftarrow{v}_p, \vec{v}_p)\}$ – set of reversion arcs, responsible for connecting the upper and the lower section of the network.
- $A = \vec{A} \cup \overleftarrow{A} \cup \overleftrightarrow{A}$ – set of all arcs.

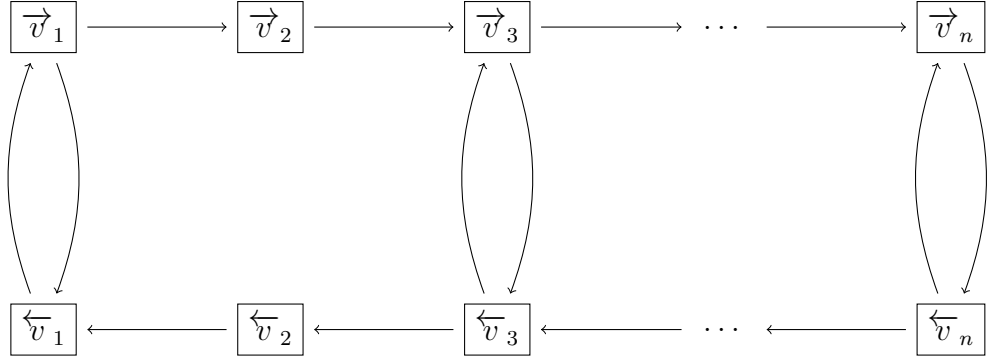


Figure 8: Representation of the network considered by the model as the directed graph G' .

This second version of the network was adopted for the following reason: since some routes can be cyclic, a single train can pass the same stopping point twice during the same trip, as each trip corresponds to completing one route. Now that we have two different vertices representing the same point, each one can be associated with one of the moments when the train passed through it. This makes it easier to determine the exact times a

train visited each point during a trip, along with the direction it was traveling. Moreover, the set C is defined to simplify the modeling of constraints that prevent collisions between vehicles. It identifies the critical points where collisions may happen.

- C — set composed of quadruples (k, q, v, a) , where $k, q, v \in V$ and $a \in A$, with $C \subseteq V \times V \times V \times A$. It contains the critical points in the network G' where collisions can occur.

Figure 9 illustrates an element $(k, q, v, a) \in C$ on a network with six stopping points, where $P = \{p_1, p_2, p_3, p_4, p_5, p_6\}$. Let $p_1, p_6 \in D$ denote depots, $p_2, p_5 \in X$ crossings, and $p_3, p_4 \in S$ stations. Therefore, p_2 and p_5 represent adjacent crossings, thus identifying a region of the network where conflicts may occur.

Let $V_2 = \{v, v'\}$ be the set of vertices that belong to crossing p_2 , where v belongs to the upper section and v' to the lower section. Let k be the vertex in the lower section of crossing p_5 . Finally, let q be the predecessor of v' , and let a denote the arc connecting q to v' .

If a vehicle departs from p_2 to the right, i.e., from v , while another vehicle departs from p_5 to the left, i.e., from k , then the segments between crossings p_2 and p_5 become a potential collision area. To avoid such conflict, the vehicle at v can only depart after the other vehicle has reached v' , assuming that the vehicle in k was the first to depart. Hence, (k, q, v, a) represents critical elements in a part of the network where collisions can occur.

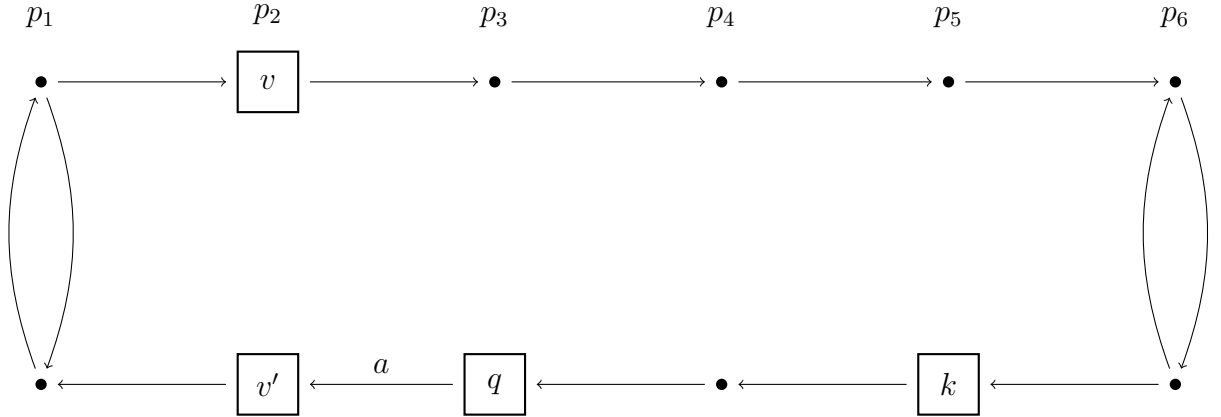


Figure 9: Quadruple $(k, q, v, a) \in C$, with v at crossing p_2 and k at crossing p_5 .

Figure 10 illustrates the same network, but now with (k, q, v, a) defined such that k belongs to crossing p_2 and v to crossing p_5 . In this case, the vertices and arcs that previously belonged to the upper section are now located in the lower section. This example shows that the vertices and arc introduced in the previous explanation may belong either to the upper or in the lower section of the graph.

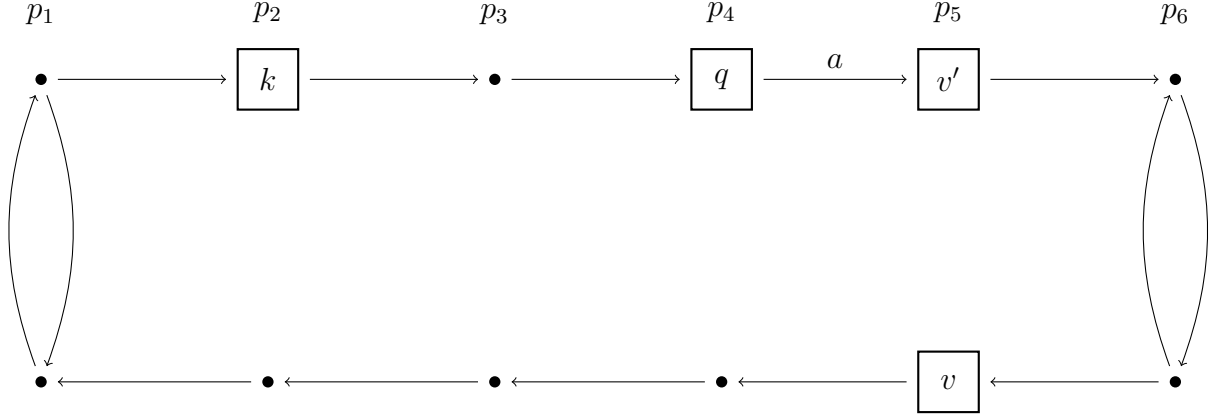


Figure 10: Quadruples $(k, q, v, a) \in C$, with v at crossing p_5 and k at crossing p_2 .

In what follows, we present the sets associated with the routes trains may complete during the day.

- R – set of all routes.
- R_{inc} – set of ordered pairs of routes (r_1, r_2) that are incompatible, i.e., route r_2 begins at a point where route r_1 does not terminate.
- R' – set of non-cyclical routes, i.e., routes that start and end at different stopping points.
- R_i^t – set of routes that train $t \in T$ can complete on trip $i \in I_t$.
- $R_{0,v}$ – set of routes that start at vertex $v \in V$.
- $R_{f,v}$ – set of routes that end at vertex $v \in V$.
- V_{dep} – set of ordered pairs of vertices (v, k) , where v is the final vertex of a route $r_1 \in R$, and k is the initial vertex of a route $r_2 \in R$, with $(r_1, r_2) \notin R_{inc}$, i.e., r_1 and r_2 are compatible routes.
- A_r – set of all arcs that belong to route $r \in R$. We denote by a_r^f the last arc of route r .

And finally, we define a set of time intervals, used to specify the demands at stations according to the time of day.

- H – set of time intervals $H = \{h_1, h_2, \dots, h_n\}$.

4.1.2 Data

We now present the data that must be considered in the mathematical formulation.

- t_a – time to travel $a \in A$.
- $\bar{t}_a$ – time to travel arc $a = (v, k) \in A$, plus the maximum dwell time that must be respected upon arriving at vertex k .
- $\hat{t}_a$ – time to travel arc $a = (v, k) \in A$, plus the minimum dwell time that must be respected upon arriving at vertex k .
- d_{vh} – demand for trains arriving at vertex $v \in V$ within time interval $h \in H$, with $d_{vh} = 0$ when $v \in \{\overleftarrow{v_0}, \overrightarrow{v_n}\}$.
- t_{max} – maximum operating time of the train line.
- α – minimum headway time.

4.1.3 Decision variables

The following decision variables are used in the mathematical model.

- x_a^{ti} – binary variable that indicates whether train $t \in T$, on trip $i \in I_t$, uses arc $a \in A$.
- $\bar{x}_{ah}^{ti}$ – binary variable that indicates whether train $t \in T$, on trip $i \in I_t$, uses arc $a \in A$, within time interval $h \in H$.
- y_v^{ti} – continuous variable that indicates the departure time of train $t \in T$, on trip $i \in I_t$, from vertex $v \in V$.
- $\bar{y}_v^{ti}$ – continuous variable that indicates the arrival time of train $t \in T$, on trip $i \in I_t$, at vertex $v \in V$.
- λ_r^{ti} – binary variable to indicate whether train $t \in T$, on trip $i \in I_t$, completes route $r \in R_i^t$.
- w_{tiv}^{ljk} – binary variable that indicates whether train $t \in T$, on trip $i \in I_t$, departs from vertex $v \in V$ after train $l \in T$, on trip $j \in I_l$, departs from vertex $k \in V$.
- u_{tiv}^{lj} – binary variable that indicates whether train $t \in T$, on trip $i \in I_t$, departs from vertex $v \in V$ after train $l \in T$, on trip $j \in I_l$.
- z – continuous variable that represents the maximum headway between any two trains.

4.1.4 Objective function and constraints

The objective function (1) aims to minimize the maximum time interval between departures from a station in a given direction, which is represented by the variable z .

$$(F1) \quad \min z \quad (1)$$

In the following, we define the constraints of the model. To begin with, the expression below (2) is responsible for obtaining the value of z .

$$z \geq (y_v^{ti} - y_v^{lj}) \quad \forall t \in T, i \in I_t, l \in T, j \in I_l, p \in S, v \in V_p \quad (2)$$

Constraints (3) establish a relation between the arcs used by a train throughout the day and those used within a specific time interval. Moreover, constraints (4) establish the relation between the arcs and the routes to which they belong.

$$x_a^{ti} = \sum_{h \in H} \bar{x}_{ah}^{ti} \quad \forall t \in T, i \in I_t, a \in A \quad (3)$$

$$x_a^{ti} = \sum_{\substack{r \in R_i^t: \\ a \in A_r}} \lambda_r^{ti} \quad \forall t \in T, i \in I_t, a \in A \quad (4)$$

Constraints (5) ensure that at most one route is assigned to each train trip, while constraints (6) prevent incompatible routes from being selected for consecutive trips.

$$\sum_{r \in R_i^t} \lambda_r^{ti} \leq 1 \quad \forall t \in T, i \in I_t \quad (5)$$

$$\lambda_{r_1}^{ti} + \lambda_{r_2}^{t,i-1} \leq 1 \quad \forall t \in T, i \in \{2, \dots, i_t\}, r_1 \in R_i^t, r_2 \in R_{i-1}^t, (r_1, r_2) \in R_{inc} \quad (6)$$

Constraints (7) ensure that a subsequent trip of a train can only be completed if the previous one was also executed, thereby maintaining the chronological order of trip indices. Furthermore, constraints (8) connect the arrival time at the final vertex of one trip to the departure time from the initial vertex of the subsequent trip of the same train, ensuring temporal consistency and proper linkage between trips.

$$\sum_{r \in R_i^t} \lambda_r^{ti} \leq \sum_{r \in R_{i-1}^t} \lambda_r^{t,i-1} \quad \forall t \in T, i \in \{2, \dots, i_t\} \quad (7)$$

$$y_k^{ti} \geq \bar{y}_v^{t,i-1} - M(2 - \sum_{r \in R_{0,k} \cap R_i^t} \lambda_r^{ti} - \sum_{r \in R_{f,v} \cap R_{i-1}^t} \lambda_r^{t,i-1}) \quad \forall t \in T, i \in \{2, \dots, i_t\}, (v, k) \in V_{dep} \quad (8)$$

Constraints (9) and (10) establish that the arrival time at a vertex equals the departure time from the preceding vertex of the route, added to the corresponding travel time.

$$\bar{y}_k^{ti} \geq y_v^{ti} + t_a - M(1 - \lambda_r^{ti}) \quad \forall t \in T, i \in I_t, r \in R_i^t, a = (v, k) \in A_r \quad (9)$$

$$\bar{y}_k^{ti} \leq y_v^{ti} + t_a + M(1 - \lambda_r^{ti}) \quad \forall t \in T, i \in I_t, r \in R_i^t, a = (v, k) \in A_r \quad (10)$$

Constraints (11) and (12) ensure that the maximum and the minimum dwell times are respected at the stopping points.

$$y_k^{ti} \geq y_v^{ti} + \hat{t}_a - M(1 - \lambda_r^{ti}) \quad \forall t \in T, i \in I_t, r \in R_i^t, a = (v, k) \in A_r \setminus \{a_r^f\}, \quad (11)$$

$$y_k^{ti} \leq y_v^{ti} + \bar{t}_a - M(1 - \lambda_r^{ti}) \quad \forall t \in T, i \in I_t, r \in R_i^t, a = (v, k) \in A_r \setminus \{a_r^f\} \quad (12)$$

Constraints (13) and (14) ensure that the maximum operating time will be respected, and set the variables y and $\bar{y}$ to zero when a vertex is not visited during a trip.

$$y_v^{ti} \leq t_{max} \sum_{a \in \delta^+(v)} x_a^{ti} \quad \forall t \in T, i \in I_t, v \in V \quad (13)$$

$$\bar{y}_v^{ti} \leq t_{max} \sum_{a \in \delta^-(v)} x_a^{ti} \quad \forall t \in T, i \in I_t, v \in V \quad (14)$$

Constraints (15) and (16) define when each time interval starts and ends, activating the $\bar{x}$ variable when a train uses an arc during the interval.

$$y_v^{ti} \geq h_0 - M(1 - \bar{x}_{ah}^{ti}) \quad \forall t \in T, i \in I_t, h = (h_0, h_f) \in H, a \in \delta^+(v) \quad (15)$$

$$y_v^{ti} \leq h_f + M(1 - \bar{x}_{ah}^{ti}) \quad \forall t \in T, i \in I_t, h = (h_0, h_f) \in H, a \in \delta^+(v) \quad (16)$$

Constraints (17) establish the demand for train departures from each station for each time interval.

$$\sum_{t \in T} \sum_{i \in I_t} \sum_{a \in \delta^+(v)} \bar{x}_{ah}^{ti} \geq d_{vh} \quad \forall v \in V, h \in H \quad (17)$$

Constraints (18)–(20) define the values of the w variables by linking them to the x variables.

$$w_{tiv}^{ljk} \leq \sum_{a \in \delta^+(v) \setminus \overleftrightarrow{A}} x_a^{ti} \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v, k \in V, v \neq k \quad (18)$$

$$w_{tiv}^{ljk} \leq \sum_{a \in \delta^+(k) \setminus \overleftrightarrow{A}} x_a^{lj} \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v, k \in V, v \neq k \quad (19)$$

$$w_{tiv}^{ljk} + w_{ljk}^{tiv} \geq \sum_{a \in \delta^+(v) \setminus \overleftrightarrow{A}} x_a^{ti} + \sum_{a \in \delta^+(k) \setminus \overleftrightarrow{A}} x_a^{lj} - 1 \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v, k \in V, v \neq k \quad (20)$$

Similarly to (18)–(20), constraints (21)–(23) are responsible for defining the values for u variables, linking them to the x variables.

$$u_{tiv}^{lj} \leq \sum_{a \in \delta^+(v)} x_a^{ti} \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v \in V \quad (21)$$

$$u_{tiv}^{lj} \leq \sum_{a \in \delta^+(v)} x_a^{lj} \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v \in V \quad (22)$$

$$u_{tiv}^{lj} + u_{ljk}^{ti} \geq \sum_{a \in \delta^+(v)} x_a^{ti} + \sum_{a \in \delta^+(v)} x_a^{lj} - 1 \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v \in V \quad (23)$$

Furthermore, constraints (24) ensure that collisions from opposite directions will not occur between vehicles. Constraints (25) prevent collisions between trains traveling in the same direction by ensuring that a train can only arrive at a stopping point if no other train is currently stopped there and about to depart in the same direction. And finally, constraints (26) ensure that the minimum headway between train departures is respected.

$$y_v^{ti} \geq y_q^{lj} + t_a - M(1 - w_{tiv}^{ljk}) \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, (k, q, v, a) \in C \quad (24)$$

$$\bar{y}_v^{ti} \geq y_v^{lj} - M(1 - u_{tiv}^{lj}) \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v \in V \quad (25)$$

$$y_v^{ti} \geq y_v^{lj} + \alpha - M(1 - u_{tiv}^{lj}) \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v \in V \quad (26)$$

Finally, (27)–(34) indicate the domain of the variables used in the formulation.

$$x_a^{ti} \in \{0, 1\} \quad \forall t \in T, i \in I_t, a \in A \quad (27)$$

$$\bar{x}_a^{ti} \in \{0, 1\} \quad \forall t \in T, i \in I_t, a \in A \quad (28)$$

$$y_v^{ti} \in \mathbb{R}_+ \quad \forall t \in T, i \in I_t, v \in V \quad (29)$$

$$\bar{y}_v^{ti} \in \mathbb{R}_+ \quad \forall t \in T, i \in I_t, v \in V \quad (30)$$

$$\lambda_v^{ti} \in \{0, 1\} \quad \forall t \in T, i \in I_t, r \in R_i^t \quad (31)$$

$$w_{tiv}^{ljk} \in \{0, 1\} \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v, k \in V, v \neq k \quad (32)$$

$$u_{tiv}^{lj} \in \{0, 1\} \quad \forall t, l \in T, t \neq l, i \in I_t, j \in I_l, v \in V \quad (33)$$

$$z \in \mathbb{R}_+ \quad (34)$$

4.2 Enumeration of routes

Since the mathematical formulation is large and computationally demanding, its performance decreases when solving more challenging instances. Hence, we now introduce an alternative exact method that is capable of dealing with a wider range of instances, based on the strategy of enumerating combinations of routes.

The TTP alone is already proven to be a complex problem and known to be NP-hard (Higgins et al., 1997; Cai and Goh, 1994; Caprara et al., 2002). When we integrate it with another element (in our case, the assignment of routes), the optimization can become an even harder process. Additional variables and even big-M constraints are often needed in such cases, and then, the mathematical formulation may become too heavy to solve the instances within a feasible execution time.

Early experiments showed that solving a single timetable with all train routes already fixed can be extremely faster than solving the entire model. Thus, we propose the following strategy: enumerate all feasible combinations of routes that can be completed for each trip of each train and provide all possibilities for the model to solve, thereby focusing on the timetabling alone. As combinations are explored and feasible solutions are obtained, we set the current best solution as the upper bound for solving the subsequent combinations. Finally, after exploring all the feasible routes' combinations, we select the one that returned the best objective value.

The enumerated combinations can vary in the number of trips that the trains complete, meaning that not all trains are required to perform the maximum number of trips allowed. This flexibility contributes to increasing the number of possible combinations to be tested. For example, if two trains are allowed up to three trips each, feasible combinations may also include sequences where trains complete only one or two.

Preliminary tests also demonstrated that most instances present less than 5% of routes' combinations that lead to feasible timetables. For this reason, we highlight the importance of filtering out as many combinations as possible that would inevitably result in infeasibilities. Furthermore, we seek to avoid solving combinations that yield identical objective function values, i.e., symmetrical combinations. In the following, we list some of the strategies that were adopted to filter the number of feasible combinations before executing the model.

- Ensuring that only compatible routes, i.e., pairs of subsequent routes $(r_1, r_2) \notin R_{inc}$, appear in adjacent positions.
- Verifying that the combination guarantees demand coverage for the entire day. Since d_{vh} represents the demand of the vertices in specific time intervals, we ensure that the combination covers the total daily demand, independent of its timing.

- Excluding symmetrical combinations by ignoring cases where a sequence of routes is merely permuted between trains.

Even with such verifications, the number of combinations can still easily become excessive. As the number of trains, trips, or routes increases, the total number of combinations can become so large that running the enumeration may take longer than solving the complete model. In more extreme cases, it can even lead to numerical overflow, making it impossible to solve the problem with this approach.

For this reason, we present in the following subsection (4.3) a MILP-based heuristic approach that is more scalable than the enumeration, can solve a good amount of instances faster, and can still find good or even optimal solutions for cases that the enumeration cannot handle.

4.3 MILP-based heuristic

The heuristic approach that we now present is highly based on the enumeration method. Knowing that most combinations executed by the enumeration method do not produce feasible timetables for most instances, we base the heuristic on the following principle: find the combinations of routes that are most likely to lead to feasible timetables.

Early experiments showed that most feasible combinations consisted mostly of cyclic routes. Not only that, but optimal solutions for most instances also consisted mainly of cyclic routes. It is known that cyclic timetables can be preferable to passengers, given that they are usually more predictable and easier to remember (Cacchiani and Toth, 2012; Robenek et al., 2016). Our study does not force the timetable to be cyclic in the sense of repeating arrival and departure time patterns at fixed intervals, but we acknowledge that implementing cyclic train trips may ease the scheduling of a more predictable timetable for passengers to follow.

The main routine of the MILP-based heuristic is described in Algorithm 1. The proposed algorithm starts by selecting a set of initial candidate combinations (Line 2). For each train in such a combination, two distinct patterns are possible.

- The train repeats the same cyclic route, starting at the initial depot, from its first to its final trip.
- The train performs a non-cyclic route in its first trip, starting at the initial depot, and then repeats a single cyclic route from its second to its final trip.

These cyclic routes are selected from what we call the set of valid routes. Initially, this set is composed of cyclic routes that visit every stopping point on the line, which

we refer to as complete cyclic routes. If no complete cyclic route exists in the instance, then all available cyclic routes are included in the set of valid routes. Once the set of initial candidate combinations is generated, the timetabling is solved for each of them, and the combination that produces the best objective value is selected as the current best combination (Line 3).

As a next step, we create subsets of the current best combination by removing trips from it (Line 6). These subsets represent all possible cases of removing the last trip assigned to a train in the current best combination. Figure 11 shows an example of subsets generated from a combination represented as matrices. Rows represent trains and columns represent trips. In this example, in the current best combination, Trains 1 and 2 are assigned to perform three trips each. In the first trip, Train 1 follows route r_1 , and in the subsequent trips, it follows route r_2 . Train 2 follows route r_3 in the first trip, and in the subsequent trips, it follows route r_4 . All possible cases of removing the last trip of each train are illustrated.

(a)		1	2	3
	t_1	r_1	r_2	r_2
	t_2	r_3	r_4	r_4

(b)		1	2	3
	t_1	r_1	r_2	—
	t_2	r_3	r_4	r_4

(c)		1	2	3
	t_1	r_1	r_2	r_2
	t_2	r_3	r_4	—

(d)		1	2	3
	t_1	r_1	r_2	—
	t_2	r_3	r_4	—

Figure 11: Set of matrices representing route combinations. Matrix (a) shows the baseline combination, while matrices (b), (c), and (d) show subsets generated from it.

All of the subsets created now correspond to the current set of candidate combinations (Line 9), and sequentially, we execute the next iteration, again optimizing the timetable for all candidates (Line 10). This procedure stops when the current best combination cannot be improved after solving the new candidates, or when the subsets generated are not capable of meeting the daily demands.

Finally, as a final step, we allow the model to modify the best combination found during the procedure (Line 12). That is, the last route assigned to each train in the best combination is no longer fixed. By leaving these trips unfixed, the model can select a new route for the last trip if the solution can still be improved, or leave it empty, in which case the train will not perform any route on that trip.

Algorithm 1: MILP-BasedHeuristic()

```
1  $s^* \leftarrow \text{NULL}; f^* \leftarrow \infty$ 
2  $C \leftarrow$  Define set of initial candidate combinations
3  $s', C \leftarrow \text{SolveInitialCandidates}(C)$ 
4 while  $s'$  feasible and  $f(s') < f^*$  do
5    $s^* \leftarrow s'; f^* \leftarrow f(s')$ 
6   Create subsets from  $s^*$ 
7   if subsets do not meet the demands then
8     break
9   Update  $C$  with subsets
10   $s' \leftarrow \text{SolveCandidateCombinations}(C)$ 
11 if  $s^*$  feasible then
12   Solve model with final trips from  $s^*$  unfixed
13 return  $s^*$ 
```

It is possible that no feasible solution is found after solving the initial set of candidate combinations. Algorithm 2 describes the routine of solving the initial candidates and verifying whether a feasible solution was found. If all combinations tested were infeasible, strategies are applied to modify the set of initial candidates. Initially, we create subsets from each candidate to form a new initial set (Line 3), and attempt to solve every new combination. If no solution is found, we modify the set of valid routes that was used to generate the initial combinations (Line 9). Originally, only complete cyclic routes were considered, but we now expand the set to include all available cyclic routes, updating the initial candidates accordingly. If a feasible solution still cannot be found after exploring the new candidates, we unfix all final trips of all trains for each candidate and attempt to solve the resulting combinations (Line 12). At any stage, if a feasible solution is found, we proceed with the procedure previously described in Algorithm 1, creating subsets from the current best combination until no further improvements can be made or the daily demands are no longer met.

Algorithm 2: SolveInitialCandidates(C)

```
1  $s' \leftarrow \text{SolveCandidateCombinations}(C)$ 
2 if  $s'$  infeasible then
3   Create subsets from  $C$ 
4   if subsets meet the demands then
5     Update  $C$  with subsets
6      $s' \leftarrow \text{SolveCandidateCombinations}(C)$ 
7     if  $s'$  infeasible then
8       Expand set of valid routes ▷ Including all cyclic routes
9       Update  $C$  considering the new valid routes
10       $s' \leftarrow \text{SolveCandidateCombinations}(C)$ 
11      if  $s'$  infeasible then
12        Update  $C$  with final trips unfixed
13         $s' \leftarrow \text{SolveCandidateCombinations}(C)$ 
14    else
15      go to Line 8.
16 return  $s', C$ 
```

Furthermore, Algorithm 3 describes the execution of all candidates, selecting the best solutions found among them. It is worth noting that the TTP is a highly symmetrical problem, meaning that different combinations may lead to the same result. However, after unfixing the last trips of trains in symmetrical combinations, these combinations can yield different objective values. To ensure better solutions, we apply a tie-breaker when selecting the best combination: among a set of symmetrical combinations, we choose the one with the minimum number of distinct routes (Line 8).

Algorithm 3: SolveCandidateCombinations(C)

```
1  $s' \leftarrow \text{NULL}; f' \leftarrow \infty$ 
2 foreach  $c$  in  $C$  do
3    $s \leftarrow \text{Solve model with fixed routes using } c$ 
4   if  $s$  feasible then
5     if  $f(s) < f'$  then
6        $s' \leftarrow s; f' \leftarrow f(s)$ 
7     else if  $f(s) = f'$  then
8       Apply tie-breaker ▷ Symmetry was detected
9 return  $s'$ 
```

Also, we highlight that, just like in the enumeration method, an upper bound is set for the solver as improvements are made on the current best solution. Moreover, verifications are performed before passing the combinations to the model, allowing us to

filter out as many infeasibilities and symmetries as possible before solving the timetabling.

5 Computational experiments

The current section presents the computational results obtained after performing extensive experiments in order to verify the efficiency of the methods presented in Section 4. All implementations were developed in the C++ programming language. The experiments were performed on an Intel Xeon Silver 4316 processor, 2.30 GHz, with 125 GB of RAM, and CPLEX 22.1.1 was used as the MILP solver.

For all methods, a time limit of 12 hours was established. However, for both the enumeration and heuristic, it was also necessary to define a time limit for the execution of each individual combination. For the enumeration, the limit was set to 1 hour, while for the heuristic, it was set to 20 minutes. Furthermore, the execution of each instance was set to run on a single thread.

5.1 Instances

The instances were generated automatically and divided into the following sets based on the number of stopping points: 5–9 (from 5 to 9 points), 10–14, 15–19, and 20–24, with 20 instances in each set. For all values of stopping points in the 5–9 set, there is one instance for each number of trains, ranging from 2 to 5. For the other sets, the same is done with the number of trains ranging from 3 to 6.

The remaining parameters are randomly generated based on the predefined number of stopping points and trains. These include the number of trips each train can perform, time intervals and their bounds, the classification of points (stations, crossings, and depots), route patterns, dwell times, arc travel times, station demands, the maximum operating time, and the minimum headway.

Depots, crossings, and stations are randomly assigned within the network, with the condition that both endpoints must be depots. The initial depot is also randomly defined. The maximum operating time is selected from the interval $[3600, 64800]$. Once this value is defined, the planning horizon is evenly divided into time intervals.

Arc travel times are determined based on the maximum operating time and the size of the network. For larger networks with shorter horizons, smaller travel times are imposed, whereas for smaller networks with longer horizons, larger travel times can be assigned. The same dependency is considered when generating dwell times, headways, demands, and the maximum number of trips per train.

The selection of predefined route patterns also considers the network configuration. Since instances always have two depots located at the endpoints, the routes covering the full network and linking the two endpoints are guaranteed. When additional depots are present, routes linking them with the other depots are also included.

Moreover, a set of real-world instances was developed based on the existing scenario at the CBTU train line. These instances vary in the number of trains, number of predefined routes, and number of time intervals. The configuration of the network is the same for all of them, since the network of the CBTU train line is considered, and demands, minimum and maximum dwell times are also established according to the company's requirements.

5.2 Results

The results obtained for all sets of instances are registered in Tables 2–6. The instances follow a specific naming convention, where each letter indicates the corresponding feature. For example, **t2p5r2h1** corresponds to an instance with 2 trains, 5 stopping points, 2 routes, and 1 time interval.

For each instance, the tables register the following information for each method: the upper bound obtained, the solving time in seconds, and the gap relative to the best upper bound among the three methods. Additionally, for the model, the tables include the lower bound and the corresponding gap relative to this bound returned by the solver. The gap relative to the lower bound is denoted as Gap , and the gap relative to the best upper bound is denoted as Gap^* . Equations (35) and (36) define how these values are calculated, with UB^* being the best upper bound among the methods, UB the upper bound obtained by the current method, and LB the lower bound obtained by the model. Moreover, the tables also show the total number of combinations tested by the enumeration method.

$$Gap^* = \frac{UB - UB^*}{UB^*} \times 100 \quad (35)$$

$$Gap = \frac{UB - LB}{UB} \times 100 \quad (36)$$

In the tables, executions that found the optimal solution are highlighted in bold. Executions that not only identified the optimal value but also proved the optimality of the instance are underlined. For the model, this occurs when the gap relative to the lower bound reaches zero, whereas for the enumeration, it corresponds to when all combinations are explored and no execution is interrupted by the time limit.

Table 2: Instances from 5 to 9 stopping points.

Instances	#comb.	Model					Enumeration of routes			Heuristic		
		LB	UB	Time (s)	Gap (%)	Gap* (%)	UB	Time (s)	Gap* (%)	UB	Time (s)	Gap* (%)
t2p5r2h1	3.60×10^1	<u>31922.00</u>	<u>31922</u>	<u>1.07</u>	<u>0.00</u>	<u>0.00</u>	<u>31922</u>	<u>0.41</u>	<u>0.00</u>	31922	1.28	0.00
t2p6r7h3	4.62×10^3	<u>42077.00</u>	<u>42077</u>	<u>10.36</u>	<u>0.00</u>	<u>0.00</u>	<u>42077</u>	<u>2.32</u>	<u>0.00</u>	42077	0.79	0.00
t2p7r8h1	4.78×10^6	<u>17570.00</u>	<u>17570</u>	<u>5.24</u>	<u>0.00</u>	<u>0.00</u>	<u>17570</u>	<u>8.73</u>	<u>0.00</u>	17570	2.07	0.00
t2p8r2h2	1.60×10^1	<u>20077.00</u>	<u>20077</u>	<u>3.30</u>	<u>0.00</u>	<u>0.00</u>	<u>20077</u>	<u>0.79</u>	<u>0.00</u>	20077	1.19	0.00
t2p9r2h3	1.00×10^2	<u>37235.00</u>	<u>37235</u>	<u>26.25</u>	<u>0.00</u>	<u>0.00</u>	<u>37235</u>	<u>5.07</u>	<u>0.00</u>	37235	6.88	0.00
t3p5r7h1	6.86×10^3	<u>7730.00</u>	<u>7730</u>	<u>46.00</u>	<u>0.00</u>	<u>0.00</u>	<u>7730</u>	<u>11.91</u>	<u>0.00</u>	7730	8.77	0.00
t3p6r2h1	3.43×10^2	21715.69	23267	43200.21	6.67	0.00	23267	4493.70	0.00	23267	3095.69	0.00
t3p7r6h3	2.70×10^4	<u>31410.00</u>	<u>31410</u>	<u>110.87</u>	<u>0.00</u>	<u>0.00</u>	<u>31410</u>	<u>3.04</u>	<u>0.00</u>	31410	2.83	0.00
t3p8r7h1	1.64×10^6	<u>8473.00</u>	<u>8473</u>	<u>25.63</u>	<u>0.00</u>	<u>0.00</u>	<u>8473</u>	<u>5.73</u>	<u>0.00</u>	8473	10.01	0.00
t3p9r7h3	3.43×10^2	<u>19385.00</u>	<u>19385</u>	<u>24.66</u>	<u>0.00</u>	<u>0.00</u>	<u>19385</u>	<u>4.61</u>	<u>0.00</u>	19385	3.19	0.00
t4p5r7h2	6.78×10^9	21346.00	26287	43200.03	18.80	7.35	24626	28193.80	0.56	24488	2404.77	0.00
t4p6r7h1	7.97×10^{12}	8444.00	37817	43200.05	77.67	20.80	—	—	—	31305	3612.30	0.00
t4p7r7h4	2.14×10^7	31050.00	36082	43200.02	13.95	0.00	36082	<u>40.61</u>	<u>0.00</u>	36082	17.52	0.00
t4p8r2h1	2.56×10^2	16978.01	17313	43200.01	1.93	0.00	17313	4593.12	0.00	17313	3135.51	0.00
t4p9r2h1	8.10×10^1	<u>9036.00</u>	<u>9036</u>	<u>919.38</u>	<u>0.00</u>	<u>0.00</u>	<u>9036</u>	<u>188.46</u>	<u>0.00</u>	9036	692.45	0.0
t5p5r2h5	2.43×10^2	37282.00	37451	43200.01	0.45	0.00	37451	3606.85	0.00	37451	2402.33	0.00
t5p6r6h1	3.18×10^{10}	10600.00	18531	43200.02	42.80	11.43	16630	43200.00	0.00	16630	3925.78	0.00
t5p7r2h2	7.78×10^3	16782.00	25532	43200.04	34.27	10.67	23070	3622.45	0.00	29763	4809.74	29.01
t5p8r8h3	1.19×10^7	<u>17468</u>	<u>17468</u>	<u>7944.75</u>	<u>0.00</u>	<u>0.00</u>	17468	3860.97	0.00	17468	2404.16	0.00
t5p9r2h3	2.43×10^2	15485.00	15934	43200.02	2.82	0.00	15934	3620.24	0.00	15934	2404.89	0.00

Table 3: Instances from 10 to 14 stopping points.

Instances	#comb.	Model					Enumeration of routes			Heuristic		
		LB	UB	Time (s)	Gap (%)	Gap* (%)	UB	Time (s)	Gap* (%)	UB	Time (s)	Gap* (%)
t3p10r6h3	2.70×10^4	<u>37611.00</u>	<u>37611</u>	<u>426.30</u>	<u>0.00</u>	<u>0.00</u>	<u>37611</u>	<u>3.47</u>	<u>0.00</u>	37611	5.61	0.00
t3p11r2h1	3.43×10^2	<u>38039.00</u>	<u>38039</u>	<u>21157.04</u>	<u>0.00</u>	<u>0.00</u>	38039	3610.97	0.00	38039	2410.09	0.00
t3p12r2h3	8.00×10^0	<u>36984.00</u>	<u>36984</u>	<u>33.95</u>	<u>0.00</u>	<u>0.00</u>	<u>36984</u>	<u>1.44</u>	<u>0.00</u>	36984	2.47	0.00
t3p13r12h2	5.93×10^4	<u>17968.00</u>	<u>17968</u>	<u>1598.56</u>	<u>0.00</u>	<u>0.00</u>	<u>17968</u>	<u>87.19</u>	<u>0.00</u>	17968	45.52	0.00
t3p14r8h5	1.42×10^7	29077.00	39947	43200.02	27.21	26.43	31595	43203.80	0.00	31595	60.83	0.00
t4p10r6h3	8.10×10^5	<u>27619.00</u>	<u>27619</u>	<u>8252.29</u>	<u>0.00</u>	<u>0.00</u>	<u>27619</u>	<u>209.53</u>	<u>0.00</u>	27619	410.06	0.00
t4p11r2h2	1.30×10^3	32395.00	40927	43200.02	20.85	0.00	46277	12101.73	13.07	46277	4813.19	13.07
t4p12r2h2	2.56×10^2	33784.00	34230	43200.12	1.30	0.00	34230	3609.64	0.00	34230	2409.00	0.00
t4p13r8h1	4.10×10^7	27614.36	37739	43201.16	26.83	19.63	31548	3706.48	0.00	31548	2409.35	0.00
t4p14r2h4	2.56×10^2	<u>42727.00</u>	<u>42727</u>	<u>10319.54</u>	<u>0.00</u>	<u>0.00</u>	<u>42727</u>	<u>621.64</u>	<u>0.00</u>	42727	854.19	0.00
t5p10r2h4	3.13×10^3	24566.16	31789	43200.03	22.72	0.00	34988	3678.23	10.06	34971	2427.35	10.01
t5p11r8h3	3.28×10^4	<u>38459.00</u>	<u>38459</u>	<u>537.17</u>	<u>0.00</u>	<u>0.00</u>	<u>38459</u>	<u>46.95</u>	<u>0.00</u>	38459	23.23	0.00
t5p12r2h2	2.43×10^2	24177.85	25186	43200.07	4.01	0.00	26296	3603.98	4.41	31622	2405.26	25.55
t5p13r2h1	7.78×10^3	31584.00	—	—	—	—	—	—	—	—	—	—
t5p14r2h1	3.20×10^1	<u>17305.00</u>	<u>17305</u>	<u>361.10</u>	<u>0.00</u>	<u>0.00</u>	<u>17305</u>	<u>60.34</u>	<u>0.00</u>	17305	384.50	0.00
t6p10r6h2	4.67×10^4	19116.00	21760	43200.01	12.15	0.00	21760	3633.49	0.00	21760	2405.03	0.00
t6p11r15h2	1.14×10^7	14558.00	18908	43200.02	23.01	0.00	18908	3846.17	0.00	18908	2406.14	0.00
t6p12r10h2	1.00×10^6	18419.00	23911	43200.04	22.97	0.00	23911	3741.73	0.00	25364	2407.20	6.08
t6p13r2h2	6.40×10^1	18195.00	23975	43200.03	24.11	0.00	24135	3606.83	0.67	26881	2408.35	12.12
t6p14r2h1	6.40×10^1	17505.00	23632	43200.27	25.93	0.00	23632	3606.88	0.00	34879	5483.81	47.59

Table 4: Instances from 15 to 19 stopping points.

Instances	#comb.	Model					Enumeration of routes			Heuristic		
		LB	UB	Time (s)	Gap (%)	Gap* (%)	UB	Time (s)	Gap* (%)	UB	Time (s)	Gap* (%)
t3p15r13h2	6.89×10^4	<u>31926.00</u>	<u>31926</u>	<u>1405.78</u>	<u>0.00</u>	<u>0.00</u>	<u>31926</u>	<u>4.04</u>	<u>0.00</u>	31926	17.51	0.00
t3p16r2h1	8.00×10^0	<u>13578.00</u>	<u>13578</u>	<u>77.47</u>	<u>0.00</u>	<u>0.00</u>	<u>13578</u>	<u>5.86</u>	<u>0.00</u>	13578	102.23	0.00
t3p17r6h3	2.70×10^4	<u>25403.00</u>	<u>25403</u>	<u>3735.12</u>	<u>0.00</u>	<u>0.00</u>	<u>25403</u>	<u>8.63</u>	<u>0.00</u>	25403	35.21	0.00
t3p18r2h4	6.40×10^1	<u>31462.00</u>	<u>31462</u>	<u>8044.86</u>	<u>0.00</u>	<u>0.00</u>	<u>31462</u>	<u>58.03</u>	<u>0.00</u>	31462	133.26	0.00
t3p19r2h5	6.40×10^1	<u>41170.00</u>	<u>41170</u>	<u>2208.85</u>	<u>0.00</u>	<u>0.00</u>	<u>41170</u>	<u>91.73</u>	<u>0.00</u>	41170	399.35	0.00
t4p15r2h1	1.30×10^3	<u>34198.00</u>	<u>42294</u>	<u>43200.83</u>	19.14	16.29	<u>36380</u>	<u>3675.38</u>	0.03	<u>36369</u>	<u>2481.12</u>	0.00
t4p16r2h2	2.56×10^2	<u>27903.00</u>	<u>30625</u>	<u>43200.06</u>	8.89	0.60	<u>30590</u>	<u>3628.44</u>	0.48	<u>30443</u>	<u>2447.97</u>	0.00
t4p17r11h3	1.46×10^4	<u>38009.00</u>	<u>38009</u>	<u>1472.22</u>	<u>0.00</u>	<u>0.00</u>	<u>38009</u>	<u>52.94</u>	<u>0.00</u>	38009	155.73	0.00
t4p18r2h4	2.56×10^2	<u>39382.00</u>	<u>39382</u>	<u>34175.85</u>	<u>0.00</u>	<u>0.00</u>	<u>39382</u>	<u>252.09</u>	<u>0.00</u>	39382	789.13	0.00
t4p19r15h1	3.91×10^5	<u>5008.00</u>	<u>5008</u>	<u>2988.64</u>	<u>0.00</u>	<u>0.00</u>	<u>5008</u>	<u>59.56</u>	<u>0.00</u>	5008	236.01	0.00
t5p15r2h2	3.20×10^1	<u>22819.00</u>	<u>24939</u>	<u>43200.15</u>	8.50	0.00	<u>24939</u>	<u>3615.71</u>	0.00	<u>25004</u>	<u>2419.23</u>	0.26
t5p16r15h2	7.59×10^5	<u>15836.00</u>	<u>18440</u>	<u>43200.05</u>	14.12	0.00	<u>18440</u>	<u>4199.12</u>	0.00	<u>18440</u>	<u>2423.33</u>	0.00
t5p17r6h1	7.78×10^3	<u>17797.88</u>	<u>22620</u>	<u>43200.04</u>	21.32	0.00	<u>22620</u>	<u>3774.69</u>	0.00	<u>23034</u>	<u>2425.76</u>	1.83
t5p18r6h1	7.78×10^3	<u>17834.00</u>	<u>17834</u>	<u>8886.44</u>	<u>0.00</u>	<u>0.00</u>	17834	3764.70	0.00	17834	2427.71	0.00
t5p19r16h1	1.05×10^6	<u>26183.00</u>	<u>26183</u>	<u>6837.93</u>	<u>0.00</u>	<u>0.00</u>	<u>26183</u>	<u>1785.16</u>	<u>0.00</u>	26183	1687.57	0.00
t6p15r7h1	1.18×10^5	24638.00	29656	43200.03	16.92	0.00	<u>29656</u>	<u>5094.36</u>	<u>0.00</u>	29656	3545.80	0.00
t6p16r6h1	4.67×10^4	<u>17836.00</u>	—	—	—	—	<u>43922</u>	<u>15647.16</u>	0.00	<u>43922</u>	<u>6027.29</u>	0.00
t6p17r10h1	1.00×10^6	<u>12694.00</u>	<u>15137</u>	<u>43200.02</u>	16.14	1.50	<u>14913</u>	<u>3808.74</u>	0.00	<u>15782</u>	<u>2433.12</u>	5.83
t6p18r15h3	1.14×10^7	<u>37575.00</u>	<u>58279</u>	<u>43200.01</u>	35.53	0.11	<u>58216</u>	<u>707.04</u>	<u>0.00</u>	58216	2147.72	0.00
t6p19r2h1	6.40×10^1	<u>26751.00</u>	—	—	—	—	<u>44638</u>	<u>8503.63</u>	0.00	<u>44638</u>	<u>4678.17</u>	0.00

Table 5: Instances from 20 to 24 stopping points.

Instances	#comb.	Model					Enumeration of routes			Heuristic		
		LB	UB	Time (s)	Gap (%)	Gap* (%)	UB	Time (s)	Gap* (%)	UB	Time (s)	Gap* (%)
t3p20r16h2	1.52×10^6	<u>29645.00</u>	<u>29645</u>	<u>4917.54</u>	<u>0.00</u>	<u>0.00</u>	<u>29645</u>	<u>18.15</u>	<u>0.00</u>	29645	31.50	0.00
t3p21r13h5	2.16×10^5	<u>46690.00</u>	<u>46690</u>	<u>6423.94</u>	<u>0.00</u>	<u>0.00</u>	<u>46690</u>	<u>175.31</u>	<u>0.00</u>	46690	99.61	0.00
t3p22r6h1	2.74×10^3	<u>17667.00</u>	<u>17667</u>	<u>3895.35</u>	<u>0.00</u>	<u>0.00</u>	<u>17667</u>	<u>150.26</u>	<u>0.00</u>	17667	468.97	0.00
t3p23r12h4	5.93×10^4	<u>39934.00</u>	<u>39934</u>	<u>9640.18</u>	<u>0.00</u>	<u>0.00</u>	<u>39934</u>	<u>5075.94</u>	<u>0.00</u>	39934	182.41	0.00
t3p24r7h5	6.86×10^3	<u>49320.00</u>	<u>49320</u>	<u>12579.82</u>	<u>0.00</u>	<u>0.00</u>	<u>49320</u>	<u>648.06</u>	<u>0.00</u>	49320	230.84	0.00
t4p20r8h1	4.10×10^3	<u>22495.00</u>	<u>22495</u>	<u>2673.40</u>	<u>0.00</u>	<u>0.00</u>	<u>22495</u>	<u>66.06</u>	<u>0.00</u>	22495	554.50	0.00
t4p21r8h2	4.10×10^3	<u>25530.00</u>	<u>25530</u>	<u>3330.71</u>	<u>0.00</u>	<u>0.00</u>	<u>25530</u>	<u>247.78</u>	<u>0.00</u>	<u>25770</u>	<u>1486.33</u>	0.94
t4p22r2h1	2.56×10^2	<u>36860.00</u>	<u>49983</u>	<u>43200.03</u>	26.26	5.59	<u>47337</u>	<u>3631.98</u>	0.00	<u>49826</u>	<u>4430.59</u>	5.26
t4p23r10h2	1.00×10^4	<u>26360.00</u>	<u>26360</u>	<u>14022.07</u>	<u>0.00</u>	<u>0.00</u>	<u>26360</u>	<u>595.36</u>	<u>0.00</u>	26360	1093.16	0.00
t4p24r2h1	1.60×10^1	<u>16180.00</u>	<u>16180</u>	<u>5153.22</u>	<u>0.00</u>	<u>0.00</u>	<u>16180</u>	<u>908.56</u>	<u>0.00</u>	16180	1762.65	0.00
t5p20r2h1	2.43×10^2	<u>22096.00</u>	<u>27539</u>	<u>43200.44</u>	19.76	0.00	<u>36222</u>	<u>7214.54</u>	31.52	<u>46135</u>	<u>3631.26</u>	67.53
t5p21r2h2	3.20×10^1	<u>18320.00</u>	<u>19450</u>	<u>43200.02</u>	5.81	0.00	<u>19450</u>	<u>3614.31</u>	0.00	<u>19450</u>	<u>2434.72</u>	0.00
t5p22r13h2	3.71×10^5	13956.00	29058	43200.01	51.97	0.00	<u>29058</u>	<u>3708.20</u>	<u>0.00</u>	29058	2965.95	0.00
t5p23r2h1	3.20×10^1	<u>34690.00</u>	<u>34690</u>	<u>14772.95</u>	<u>0.00</u>	<u>0.00</u>	<u>34690</u>	<u>511.95</u>	<u>0.00</u>	34690	1718.22	0.00
t5p24r2h2	3.20×10^1	<u>24498.00</u>	<u>33006</u>	<u>43200.02</u>	25.78	0.00	<u>33006</u>	<u>3619.98</u>	0.00	<u>33006</u>	<u>2442.35</u>	0.00
t6p20r2h2	6.40×10^1	<u>16671.00</u>	<u>45892</u>	<u>43200.01</u>	63.67	0.00	<u>45892</u>	<u>7567.37</u>	0.00	<u>45892</u>	<u>4255.84</u>	0.00
t6p21r2h1	6.40×10^1	<u>8363.00</u>	<u>8363</u>	<u>348.32</u>	<u>0.00</u>	<u>0.00</u>	<u>8363</u>	<u>218.27</u>	<u>0.00</u>	8363	584.97	0.00
t6p22r2h1	6.40×10^1	<u>24292.00</u>	<u>32318</u>	<u>43200.02</u>	24.84	3.05	<u>31363</u>	<u>3620.20</u>	0.00	<u>31363</u>	<u>2456.82</u>	0.00
t6p23r2h2	6.40×10^1	<u>14982.00</u>	—	—	—	—	<u>20781</u>	<u>3628.29</u>	0.00	<u>21916</u>	<u>2471.06</u>	5.46
t6p24r16h1	1.68×10^7	<u>17292.00</u>	—	—	—	—	<u>27034</u>	<u>4420.34</u>	0.00	<u>27850</u>	<u>2468.07</u>	0.00

Table 6: Real-world CBTU instances.

Instances	#comb.	Model					Enumeration of routes			Heuristic		
		LB	UB	Time (s)	Gap (%)	Gap* (%)	UB	Time (s)	Gap* (%)	UB	Time (s)	Gap* (%)
t3p16r6h1	4.48×10^5	<u>40359.00</u>	<u>40359</u>	<u>18661.29</u>	<u>0.00</u>	<u>0.00</u>	<u>40359</u>	<u>1702.54</u>	<u>0.00</u>	40359	117.19	0.00
t3p16r6h4	4.48×10^5	38656.00	—	—	—	—	<u>47055</u>	<u>1714.26</u>	<u>0.00</u>	47175	147.80	0.26
t3p16r6h6	4.48×10^5	34361.00	51101	43200.39	32.76	8.66	<u>47030</u>	<u>1681.61</u>	<u>0.00</u>	47030	118.35	0.00
t3p16r8h1	3.31×10^8	0.00	—	—	—	—	34479	43200.4	3.48	33319	2069.45	0.00
t3p16r10h1	1.65×10^9	0.00	—	—	—	—	37419	43200.5	0.00	40359	125.49	7.86
t4p16r6h1	2.95×10^7	0.00	—	—	—	—	30519	43205.1	14.65	26619	1758.86	0.00
t4p16r6h4	2.95×10^7	6821.65	—	—	—	—	46815	43205.9	0.00	46815	924.66	0.00
t4p16r6h6	2.95×10^7	6063.71	—	—	—	—	47030	43204.8	0.00	47030	2690.01	0.00
t4p16r8h1	1.64×10^{11}	0.00	—	—	—	—	—	—	—	24060	20439.90	0.00
t4p16r10h1	2.80×10^9	9260.00	32419	43200.30	71.44	24.50	<u>26039</u>	<u>6964.77</u>	<u>0.00</u>	26199	591.69	0.61
t5p16r6h1	1.00×10^8	0.00	—	—	—	—	—	—	—	—	—	—
t5p16r6h4	1.00×10^8	0.00	—	—	—	—	—	—	—	—	—	—
t5p16r6h6	1.00×10^8	6872.20	—	—	—	—	<u>47030</u>	<u>36324.1</u>	<u>0.00</u>	47030	1246.46	0.00
t5p16r8h1	9.43×10^{11}	0.00	—	—	—	—	—	—	—	34819	43845.38	0.00
t5p16r10h1	7.48×10^{12}	0.00	—	—	—	—	—	—	—	—	—	—

It is notable that the performance of each method varies depending on the instance being solved. For some instances with a smaller number of route combinations, the enumeration method achieves better results. On the other hand, when the number of combinations increases, the heuristic can be more efficient, often reaching good solutions in most of the tested cases. For example, this behavior can be seen by comparing both the number of combinations and their performance across methods for instances such as **t2p6r7h3**, **t2p7r8h1**, **t3p16r2h1** and **t6p16r6h1**. However, since the instances include many different parameters that can make solving more challenging, it is difficult to identify a single factor that is determinant for the performance of the methods.

Furthermore, it is relevant to note that the model was able to find feasible solutions for only 3 out of the 15 real-world instances, with only one of them being the optimal solution. Both the enumeration and heuristic methods performed better: the enumeration found feasible solutions for 10 instances, proving optimality for 5 of them, while the heuristic found solutions for 12 instances, with 2 of them known to be optimal. In particular, the heuristic reduced the time required to obtain (near-)optimal solutions from hours to just a few minutes, demonstrating its effectiveness in handling the more challenging cases. Still, some instances with five trains could not be solved by any of the methods, indicating room for further improvement. Overall, the model achieved feasible solutions for 20% of the instances, the enumeration of routes for 66.67%, and the MILP-based heuristic for 80%.

Figures 12–15 visually represent some solutions obtained for the set of real-world instances. These solutions were presented to the train company, which provided positive feedback regarding their quality and potential applicability. However, the solutions have not yet been tested or implemented in real life, although prospects for their implementation are still ongoing.

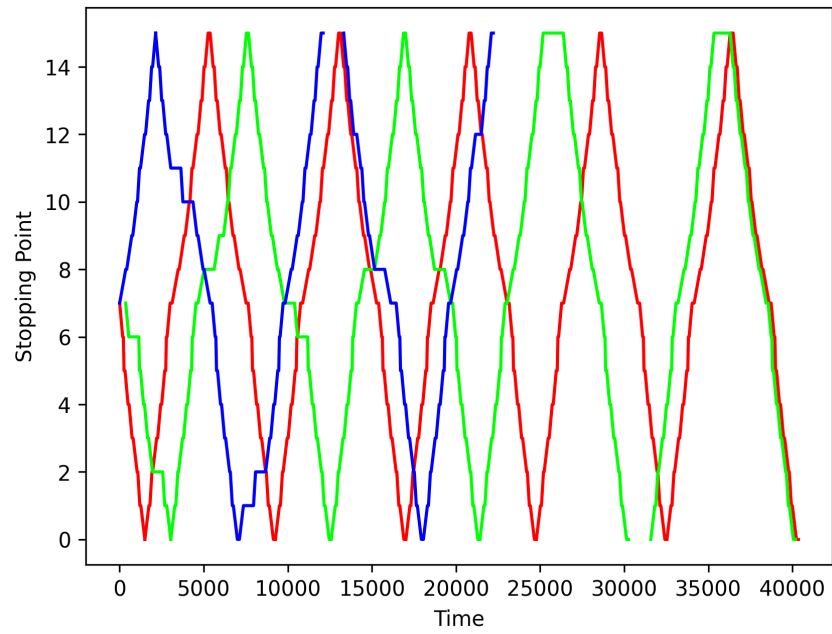


Figure 12: Real instance: t3p16r6h1

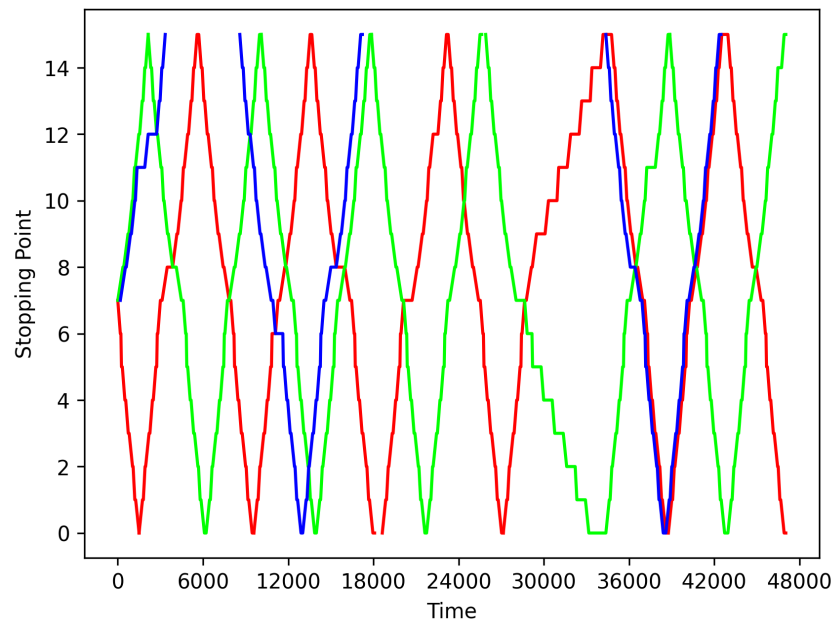


Figure 13: Real instance: t3p16r6h6

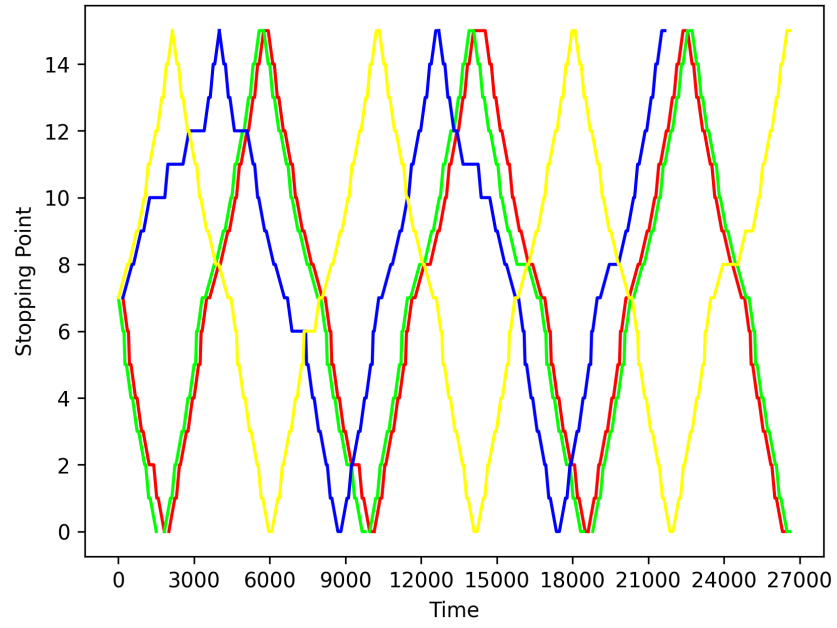


Figure 14: Real instance: t4p16r6h1

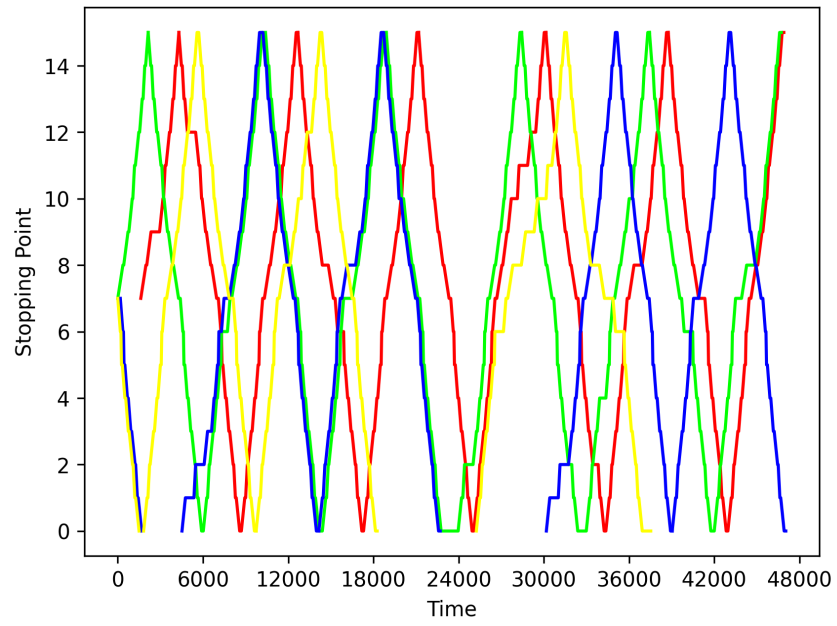


Figure 15: Real instance: t4p16r6h6

6 Concluding remarks

This work introduced a new variant of the TTP that not only addresses the timetabling of a single-track railway but also optimizes the assignment of predefined routes. A mathematical formulation was developed to address the problem, and to speed up solving the instances, an exact method based on routes enumeration, along with a MILP-based heuristic algorithm was proposed.

The computational experiments proved that most cases can be efficiently solved with the proposed methods, including real-world instances, based on the scenario at the CBTU train line in João Pessoa. However, the most suitable method may vary depending on the characteristics of each instance: some perform better with the enumeration method, others with the heuristic algorithm, and in fewer cases, even the mathematical model proves to be the most effective option. Hence, we conclude that future research should focus on conducting an instance space analysis of this problem, in order to better understand which aspects of an instance might determine the most suitable method to be used. Moreover, some improvements can still be made to the heuristic approach to ensure it can find feasible solutions for the harder instances where none of the three methods was able to provide a result, making the heuristic a robust and reliable method that can consistently deliver a feasible solution for the instances considered.

Finally, as a next step, the development of a tool that integrates these methods and provides a user-friendly interface for the train company should be considered, allowing the problem to be solved for different real-life scenarios, thereby facilitating the application of optimal solutions in practice and enhancing passenger satisfaction across different contexts.

Bibliography

- V. Cacchiani and P. Toth. Nominal and robust train timetabling problems. *European Journal of Operational Research*, 219(3):727–737, 2012. doi: 10.1016/j.ejor.2011.11.003.
- B. Szpigel. Optimal train scheduling on a single line railway. *Journal of Operations Research*, 72:344–351, 1973.
- X. Cai and C.J. Goh. A fast heuristic for the train scheduling problem. *Computers & Operations Research*, 21(5):499–510, 1994. doi: 10.1016/0305-0548(94)90099-X.
- M. Carey and D. Lockwood. A model, algorithms and strategy for train pathing. *Journal of the Operational Research Society*, 46(8):988–1005, 1995. doi: 10.1057/jors.1995.136.
- A. Higgins, E. Kozan, and L. Ferreira. Heuristic techniques for single line train scheduling. *Journal of Heuristics*, 3:43–62, 1997. doi: 10.1023/A:1009672832658.
- U. Brännlund, P. O. Lindberg, A. Nõu, and J.-E. Nilsson. Railway timetabling using lagrangian relaxation. *Transportation Science*, 32(4):358–369, 1998. doi: 10.1287/trsc.32.4.358.
- A. Caprara, M. Fischetti, and P. Toth. Modeling and solving the train timetabling problem. *Operations Research*, 50(5):851–861, 2002. doi: 10.1287/opre.50.5.851.362.
- X. Zhou and M. Zhong. Single-track train timetabling with guaranteed optimality: Branch-and-bound algorithms with enhanced lower bounds. *Transportation Research Part B: Methodological*, 41(3):320–341, 2007. doi: 10.1016/j.trb.2006.05.003.
- P. Vansteenwegen and D. Van Oudheusden. Developing railway timetables which guarantee a better service. *European Journal of Operational Research*, 173(1):337–350, 2006. doi: 10.1016/j.ejor.2004.12.013.
- C. Liebchen. Periodic timetable optimization in public transport. *Operations Research Proceedings*, 2006:29–36, 2006. doi: 10.1016/j.ejor.2004.12.013.
- E. Barrena, D. Canca, L. C. Coelho, and G. Laporte. Exact formulations and algorithm for the train timetabling problem with dynamic demand. *Computers & Operations Research*, 44:66–74, 2014. doi: 10.1016/j.cor.2013.11.003.
- T. Robenek, Y. Maknoon, S. S. Azadeh, J. Chen, and M. Bierlaire. Passenger centric train timetabling problem. *Transportation Research Part B: Methodological*, 89:107–126, 2016. doi: 10.1016/j.trb.2016.04.003.
- M. Schmidt and A. Schöbel. Timetabling with passenger routing. *OR Spectrum*, 37:75–97, 2015. doi: 10.1007/s00291-014-0360-0.

- Y. Lee and C. Y. Chen. A heuristic for the train pathing and timetabling problem. *Transportation Research Part B: Methodological*, 43(8-9):837–851, 2009. doi: 10.1016/j.trb.2009.01.009.
- Y. Sun, C. Cao, and C. Wu. Multi-objective optimization of train routing problem combined with train scheduling on a high-speed railway network. *Transportation Research Part C: Emerging Technologies*, 44:1–20, 2014. doi: 10.1016/j.trc.2014.02.023.
- Y. Yue, S. Wang, L. Zhou, L. Tong, and M. R. Saat. Optimizing train stopping patterns and schedules for high-speed passenger rail corridors. *Transportation Research Part C: Emerging Technologies*, 63:126–146, 2016. doi: 10.1016/j.trc.2015.12.007.
- V. Cacchiani, J. Qi, and L. Yang. Robust optimization models for integrated train stop planning and timetabling with passenger demand uncertainty. *Transportation Research Part B: Methodological*, 136:1–29, 2020. doi: 10.1016/j.trb.2020.03.009.
- M. Boroun, S. Ramezani, N. V. Farahani, E. Hassannayebi, S. Abolmaali, and M. Shakhbayifar. An efficient heuristic method for joint optimization of train scheduling and stop planning on double-track railway systems. *INFOR: Information Systems and Operational Research*, 58(4):652–679, 2020. doi: 10.1080/03155986.2020.1746100.
- J. Parbo, O. A. Nielsen, and C. G. Prato. Passenger perspectives in railway timetabling: A literature review. *Transport Reviews*, 36(4):500–526, 2016. doi: 10.1080/01441647.2015.1113574.